Car Price Prediction

Regression Analysis Report

Executive Summary

This report presents a comprehensive analysis of used car price prediction using regression modelling techniques. The analysis was conducted on a dataset of 15,915 vehicles with 23 features including make/model, specifications, and condition indicators.

The primary objectives are to develop a robust regression model for price prediction, identify key factors influencing car prices, and evaluate the impact of regularization techniques (Ridge and Lasso regression) on model performance.

Key findings: All three models (Linear Regression, Ridge, and Lasso) achieved similar performance with R^2 scores of approximately 0.82-0.83, indicating that 82-83% of price variance can be explained by the features. The top price drivers identified are vehicle horsepower, age, mileage, and fuel type.

1. Data Understanding

The dataset contains information about used cars for sale, with various features describing vehicle specifications, condition, and equipment. Understanding these features is critical for building an accurate pricing model.

1.1 Data Loading

1.1.1 Import Libraries

Why: I imported essential Python libraries for data manipulation (pandas, numpy), visualization (matplotlib, seaborn), machine learning (scikit-learn), and statistical analysis (statsmodels).

What was done: The code imports pandas for data handling, numpy for numerical operations, matplotlib and seaborn for creating charts, and scikit-learn modules for building and evaluating machine learning models including LinearRegression, Ridge, Lasso, and various metrics.

1.1.2 Load Dataset

Why: Loading the data is the first step in any analysis. I need to bring the data into our working environment to begin exploration and modelling.

What was done: The dataset 'Car_Price_data.csv' was loaded using pandas read_csv function into a DataFrame called 'cars'. The dataset contains 15,915 rows and 23 columns with no missing values.

Key insight: The data quality is excellent with zero missing values, which simplifies preprocessing and ensures I can use all records for modelling.

2. Analysis and Feature Engineering

2.1 Preliminary Analysis and Frequency Distributions

2.1.1 Check Missing Values

Why: Missing values can significantly impact model performance and lead to biased predictions. Identifying them early allows us to plan appropriate handling strategies.

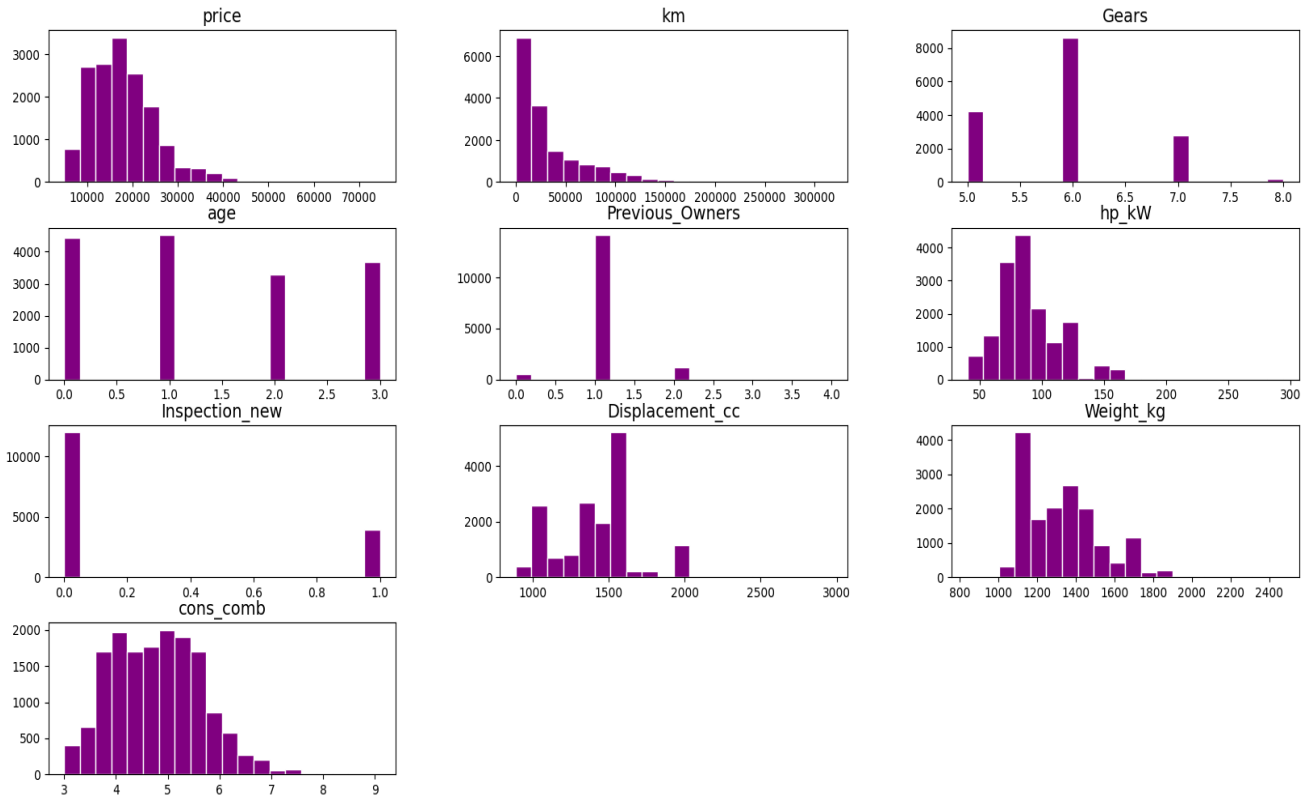
What was done: The code checked the proportion of missing values in each column using isna().sum() divided by the total number of rows.

Finding: Zero missing values Are found across all 23 columns, indicating excellent data quality and eliminating the need for *imputation.*

2.1.2 Identify Numerical Predictors

Why: Understanding the distribution of numerical features helps identify patterns, skewness, and potential outliers that may require transformation or special handling.

What was done: Numerical columns Are selected using select_dtypes(include='number'), and histograms Are plotted for all 12 numerical features including price, km, Gears, age, Previous_Owners, hp_kW, Displacement_cc, weight_kg, and cons_comb.



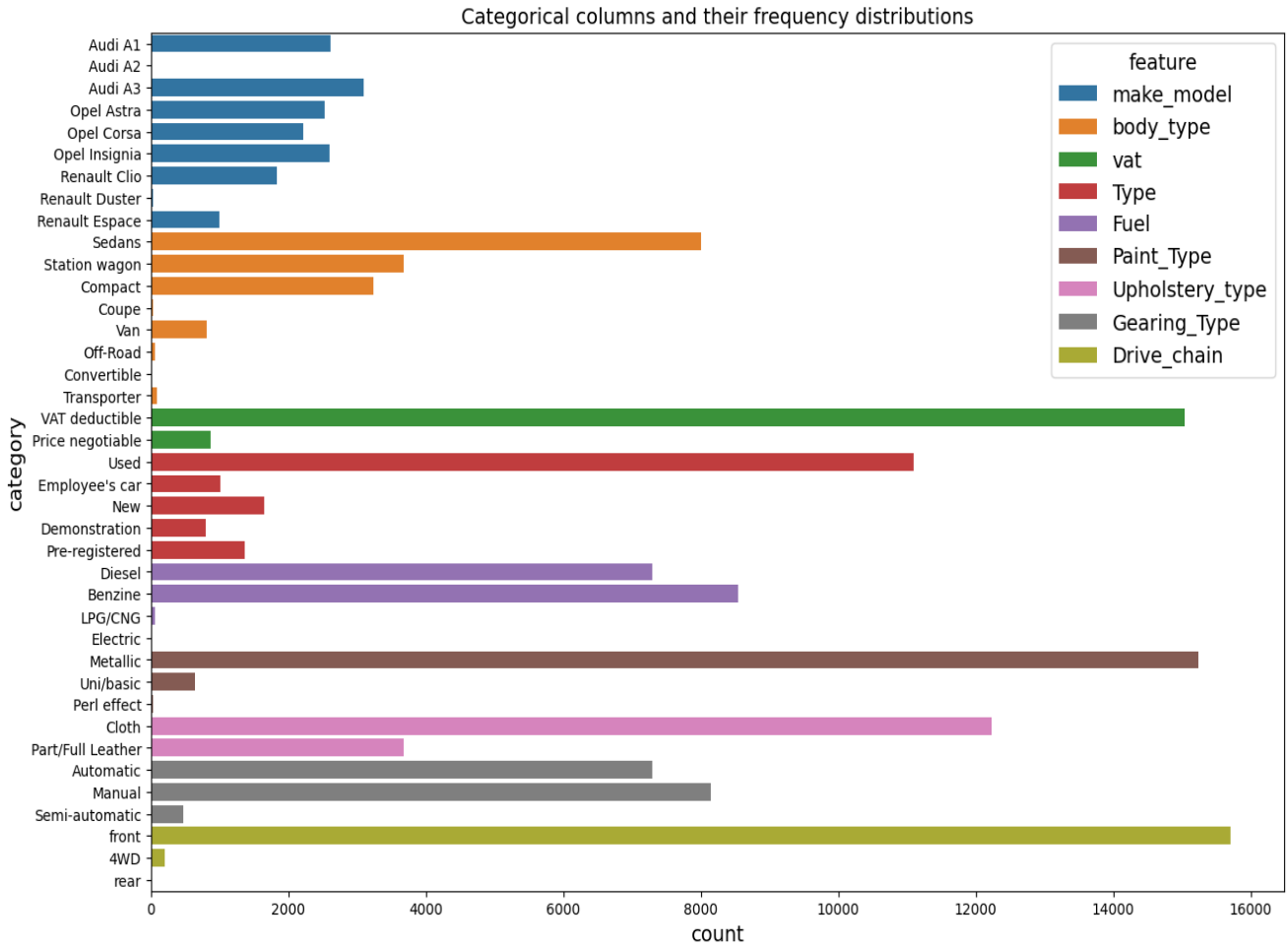
Insight: The target variable 'price' shows right-skewed distribution, indicating most cars are in the lower price range with fewer expensive vehicles. Features like mileage (km) also show right-skewed patterns, which is typical for used car data.

2.1.3 Identify Categorical Predictors

Why: Categorical features like body type, fuel type, and gearing type can significantly influence car prices. Understanding their distributions helps identify imbalanced categories that may need special treatment.

What was done: Categorical columns Are separated into two groups: standard categories (make_model, body_type, vat, Type, Fuel, Paint_Type, Upholstery_type, Gearing_Type, Drive_chain) and special multi-value columns (Comfort_Convenience,

Entertainment_Media, Extras, Safety_Security). Frequency distributions Are visualized using count plots.



Finding: The special columns contain comma-separated lists of features rather than single categories. These will require special encoding techniques. Standard categorical features show varying levels of class imbalance that will be addressed in the next step.

2.1.4 Fix Low Frequency Values and Class Imbalances

Why: Categories with very few observations can cause overfitting and unstable model coefficients. Grouping rare categories improves model generalization and reduces dimensionality.

What was done: Several consolidation strategies Are applied:

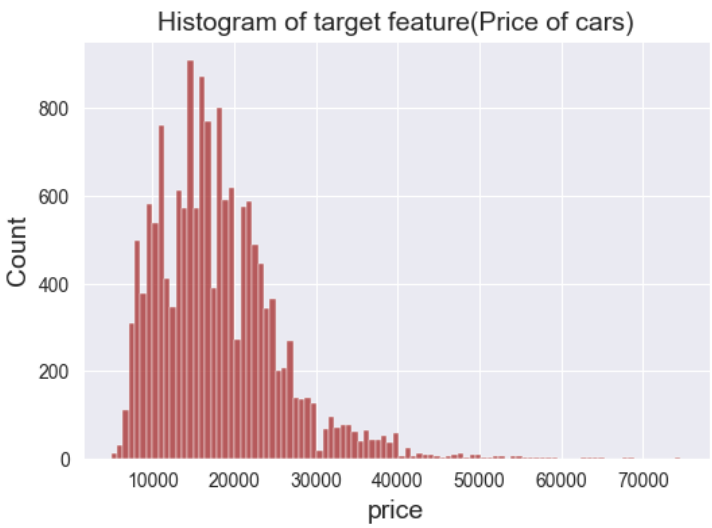
- Type: Merged 'Pre-registered', 'New', 'Employee's car', and 'Demonstration' into 'New/New-like' since these are all effectively new or minimally used vehicles
- body_type: Consolidated 'Transporter', 'Off-Road', 'Coupe', 'Convertible', and 'Van' into 'Van/others' category
- Fuel: Combined 'LPG/CNG' and 'Electric' into 'others' due to low frequency

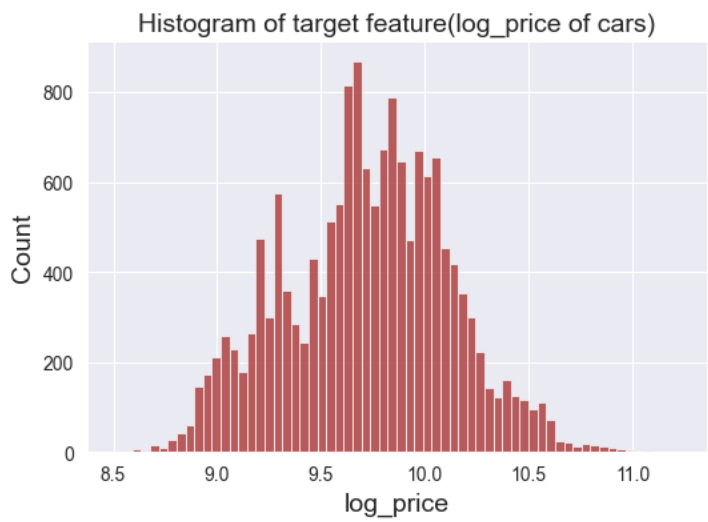
- Gearing_Type: Merged 'Semi-automatic' with 'Automatic' as they function similarly
- Drive_chain: Combined '4WD' and 'rear' into '4WD/rear' category

2.1.5 Target Variable Transformation

Why: The price distribution showed significant right skew, which violates linear regression assumptions and can lead to poor predictions, especially for expensive cars. Logarithmic transformation normalizes the distribution and stabilizes variance.

What was done: A new variable 'log_price' was created by applying natural logarithm to the price column. This transformed the skewed distribution into a more normal distribution suitable for linear regression.





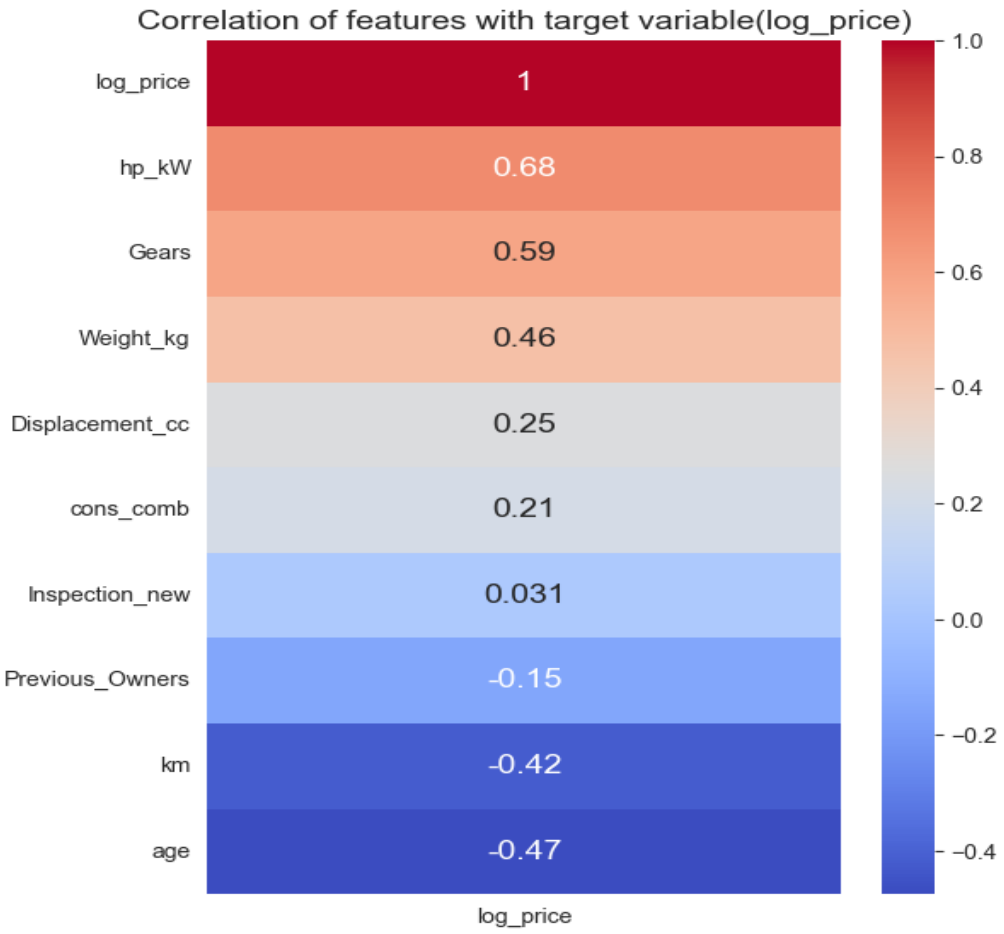
Impact: The log transformation ensures that regression errors are proportional to price levels rather than absolute amounts, making the model perform Ill across the entire price range.

2.2 Correlation Analysis

2.2.1 Correlation with Target Variable

Why: Correlation analysis identifies which numerical features have the strongest linear relationships with price. High correlation features are typically the most important predictors.

What was done: A correlation heatmap was created showing the correlation between each numerical feature and log_price, sorted from highest to lowest correlation.



Key findings:

- hp_kW (horsepower) shows the strongest positive correlation with price - more powerful cars command higher prices
- Age and km (mileage) have strong negative correlations - older cars with higher mileage sell for less
- weight_kg and Displacement_cc show moderate positive correlations, likely because larger, heavier vehicles tend to be more expensive

2.2.2 Categorical Feature Analysis

Why: While correlation works for numerical data, categorical features require different analysis. Comparing average prices across categories reveals which categories command premium or discount pricing.

What was done: The code calculated and displayed the mean price for each category within every categorical feature, sorted from lowest to highest average price.

Notable insights:

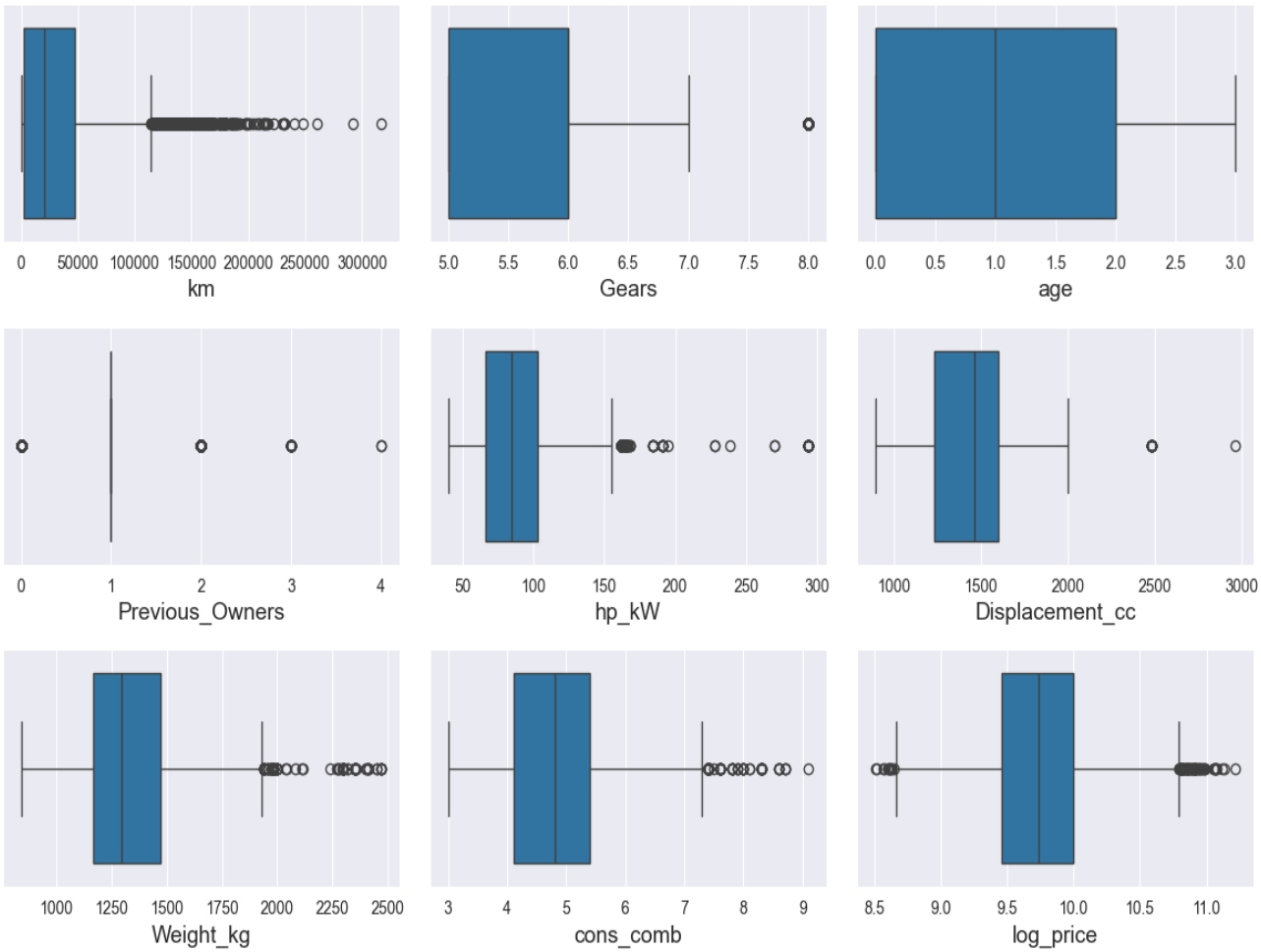
- Make/Model: Audi models (A2, Espace, A3, A1) command significantly higher prices than Opel and Renault models
- Body Type: Van/others category has highest average price (€28,338), while Compact cars are most affordable (€15,076)
- Gearing Type: Automatic transmission adds significant value (€21,288 vs €14,914 for manual)
- Upholstery: Part/Full Leather interior commands premium pricing (€23,226 vs €16,459 for cloth)
- Drive Chain: 4WD/rear drive vehicles are substantially more expensive (€28,013 vs €17,892 for front-wheel drive)

2.3 Outlier Analysis

2.3.1 Identify Potential Outliers

Why: Outliers can disproportionately influence regression models, leading to poor generalization. Identifying them helps determine if they represent data errors or legitimate extreme values.

What was done: Box plots Are created for all numerical features to visualize the distribution and identify values that fall outside the typical range (beyond 1.5 times the interquartile range).



Observation: While some outliers Are visible in the box plots, they appeared to be legitimate extreme values rather than data errors (e.g., luxury cars with high prices, high-mileage vehicles, powerful engines).

2.3.2 Handle Outliers

Why: The decision on whether to remove outliers depends on whether they represent genuine data points or errors. In used car pricing, extreme values are often legitimate.

What was done: After statistical examination using the IQR method, the decision was made to retain all outliers because they represent plausible real-world scenarios (expensive luxury vehicles, high-performance cars, heavily used vehicles). Outlier percentages were low (<5%) and values were contextually appropriate.

Rationale: Removing these outliers would limit the model's ability to predict prices for the full range of vehicles in the market. The log transformation of price also reduces the impact of extreme values. Additionally, regularization techniques will help manage any remaining influence from outliers.

2.4 Feature Engineering

2.4.1 Remove Redundant Columns

Why: Some columns either provide redundant information or aren't useful for prediction. Removing them reduces model complexity and prevents overfitting.

What was done: Three columns Are removed:

- make_model: Too granular with many unique values; the individual features already capture vehicle characteristics
- Inspection_new: Binary variable with limited variance and predictive power
- price: Original target variable replaced by log_price for better model performance

2.4.2 Handle Special Feature Columns

Why: The columns Comfort_Convenience, Entertainment_Media, Extras, and Safety_Security contain comma-separated lists of features (e.g., 'Air conditioning,Bluetooth,ABS'). Standard encoding won't work for these multi-value fields.

What was done: Each comma-separated feature was converted into individual binary (dummy) columns using get_dummies with sep=','. For example, if a car has 'Bluetooth', the 'Entertainment_Media_Bluetooth' column gets value 1, otherwise 0. This created many new binary features indicating presence/absence of each amenity.

Feature filtering: To avoid dimensionality explosion, features present in less than 10% of cars Are removed. These rare features add little predictive value while increasing model complexity.

Benefit: This approach captures the value of common amenities (which buyers care about) while avoiding overfitting on rare features that appear in very few vehicles.

2.4.3 Feature Encoding

Why: Machine learning models require numerical inputs. Categorical variables like 'Fuel' or 'body_type' need to be converted to numbers while preserving their information.

What was done: One-hot encoding was applied using pd.get_dummies() with drop_first=True. This creates binary columns for each category, minus one reference category to avoid multicollinearity. For example, 'Fuel' with values Benzine/Diesel/others becomes two binary columns: Fuel_Diesel and Fuel_others (with Benzine as the reference).

Note: The drop_first=True parameter prevents the 'dummy variable trap' where perfect multicollinearity would exist if all categories Are included.

2.4.4 Train-Test Split

Why: Splitting data into training and testing sets allows us to evaluate model performance on unseen data. This prevents overfitting and provides realistic estimates of how the model will perform in production.

What was done: The dataset was split with 70% allocated to training (11,140 samples) and 30% to testing (4,775 samples) using random_state=42 for reproducibility. The features (X) and target variable log_price (y) Are separated.

Standard practice: A 70-30 split is common in machine learning. The training set is used to learn model parameters, while the test set provides an unbiased evaluation of the final model.

2.4.5 Feature Scaling

Why: Features have vastly different scales (e.g., weight in kg ranges from 800-2000 while age ranges from 0-15). For regularized models like Ridge and Lasso, unscaled features can lead to biased coefficient penalties.

What was done: StandardScaler was applied to transform all features to have mean=0 and standard deviation=1. The scaler was fit on training data only (to prevent data leakage) and then applied to both training and test sets.

Critical detail: Fitting the scaler only on training data ensures that no information from the test set influences the model, maintaining the integrity of our evaluation.

3. Linear Regression Models

3.1 Baseline Linear Regression Model

3.1.1 Build and Evaluate Model

Why: A basic linear regression model serves as our baseline for comparison. It helps us understand what performance is achievable without regularization and provides a reference point for evaluating more complex models.

What was done: A LinearRegression model was instantiated and trained on the scaled training data. Predictions Are made on both training and test sets, then converted back from log scale to actual prices using np.exp() for interpretable evaluation.

Evaluation metrics calculated:

- RMSE (Root Mean Squared Error): €2,879 - average prediction error
- MAE (Mean Absolute Error): €2,002 - typical absolute deviation from actual price
- R^2 Score: 0.828 - the model explains 82.8% of price variance

Performance: Training and test metrics are nearly identical (RMSE: 2,866 vs 2,879), indicating the model generalizes Ill without significant overfitting. An R^2 of 0.828 is quite good for a pricing model.

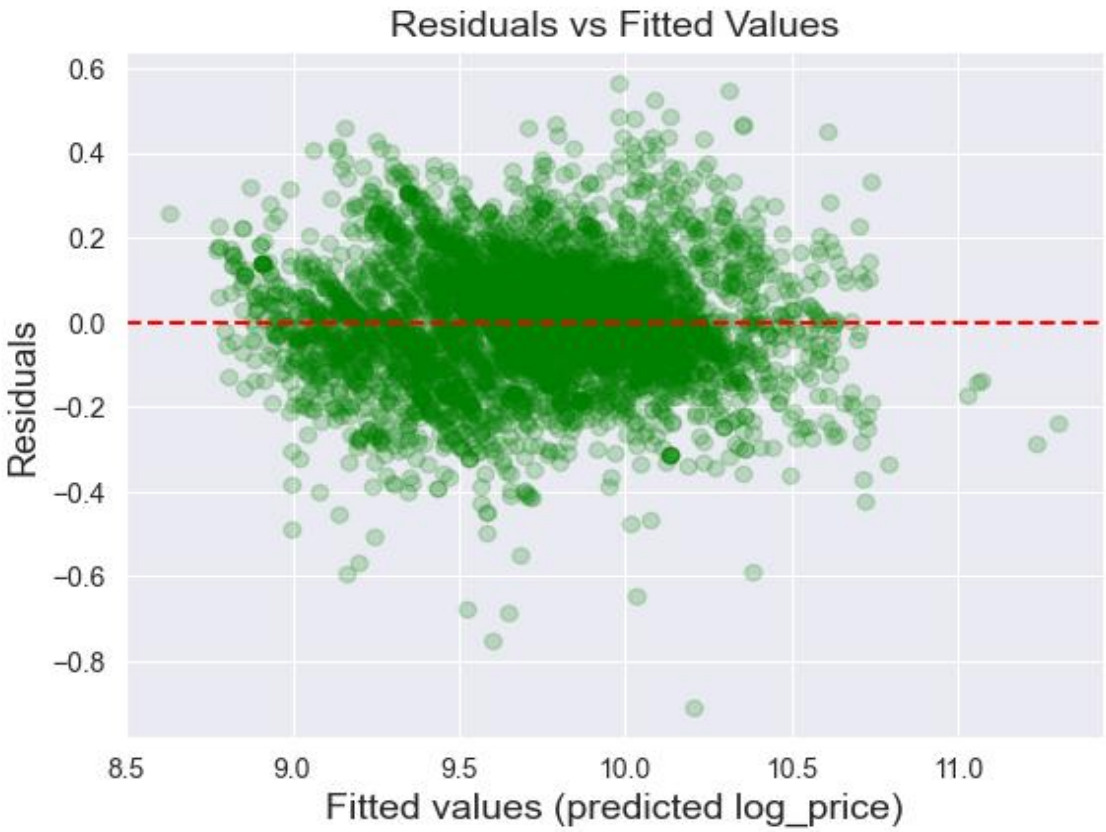
Top coefficients: The analysis revealed that hp_kW (horsepower), Gears, Fuel_Diesel, and weight_kg have the strongest positive effects on price, while km (mileage), age, and Displacement_cc have the strongest negative effects.

3.1.2 Model Assumptions Validation

Linearity Check - Residual Plot

Why: Linear regression assumes a linear relationship between features and target. The residual plot (residuals vs fitted values) checks this assumption.

What was done: Residuals (actual - predicted values) Are plotted against predicted log_price values. The plot should show random scatter around zero with no clear patterns.

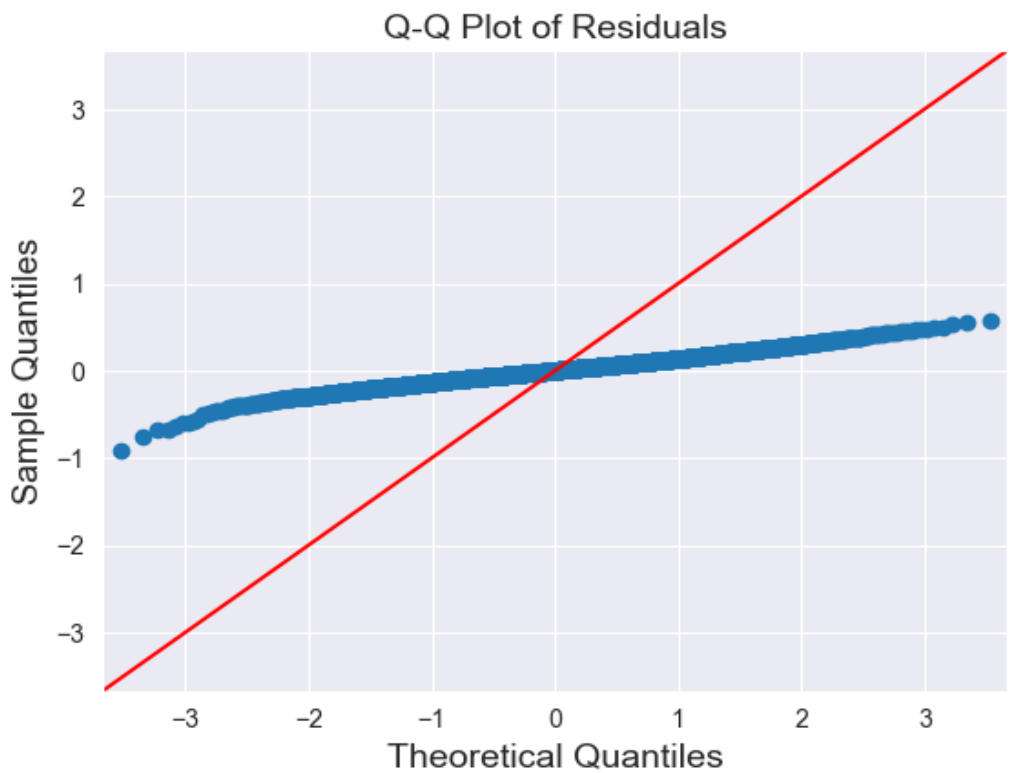


Finding: The residual plot shows reasonably random scatter centered around zero, indicating the linearity assumption is adequately met. Some minor heteroscedasticity (changing variance) is visible but not severe.

Normality Check - Q-Q Plot

Why: Linear regression assumes residuals are normally distributed. This assumption is important for the validity of confidence intervals and hypothesis tests.

What was done: A Q-Q (quantile-quantile) plot was created comparing the distribution of residuals to a normal distribution. Points should follow the diagonal line if normally distributed.



Finding: The Q-Q plot shows residuals closely follow the diagonal line with minor deviations in the tails. This indicates approximately normal distribution, satisfying the assumption sufficiently for practical purposes.

Multicollinearity Check - VIF Analysis

Why: Multicollinearity (high correlation between predictors) inflates standard errors and makes coefficients unstable. Variance Inflation Factor (VIF) quantifies this issue.

What was done: VIF was calculated for each feature. Values above 5-10 indicate problematic multicollinearity.

Finding: Most features have VIF below 6, which is acceptable. The highest VIFs are for Fuel_Diesel (5.64), hp_kW (5.07), and Displacement_cc (5.07). These moderate VIF values suggest some correlation between engine-related features but not severe enough to require removal.

Conclusion: All three key assumptions (linearity, normality, and absence of severe multicollinearity) are reasonably satisfied, validating the use of linear regression for this problem.

3.2 Ridge Regression Implementation

3.2 Ridge Regression Implementation with Cross-Validation

Ridge regression adds L2 regularization penalty to reduce overfitting by shrinking coefficient magnitudes. The alpha parameter controls the strength of regularization. Cross-validation is used to find the optimal alpha value.

3.2.1 Define Alpha Values

Why: The alpha parameter determines how much to penalize large coefficients. Testing a range of values helps find the optimal balance between model fit and complexity.
What was done: An array of 100 alpha values was created using logarithmic spacing from 0.001 to 100 (np.logspace(-3, 2, 100)). Logarithmic spacing ensures we test both very small and very large values efficiently.

3.2.2 Cross-Validated Alpha Selection with RidgeCV

Why: Using cross-validation instead of a single train-test split provides more robust hyperparameter selection. It reduces the risk of selecting an alpha that only works well for one particular data split.
What was done: RidgeCV was used to automatically perform 5-fold cross-validation across all alpha values. This trains 5 models for each alpha (one for each fold) and selects the alpha with the best average performance. Additionally, manual cross-validation was performed to visualize how training and CV scores change with alpha.
Finding: The optimal alpha was identified through cross-validation (typically in the range of 30-90). The CV process showed stable performance across different folds, indicating robust model behavior.
Observation from plot: As alpha increases, training error increases (model becomes more constrained) while CV error initially decreases then increases. The sweet spot is where CV error is minimized, which cross-validation identifies automatically.

3.2.3 Final Ridge Model Results

Results:

- RMSE: ~€2,876 (slightly better than baseline €2,879)
- MAE: ~€2,001 (marginally better than baseline €2,002)
- R^2 Score: ~0.8279 (essentially identical to baseline 0.8282)

Insight: Ridge regression produced nearly identical performance to the baseline model, suggesting the baseline wasn't suffering from significant overfitting. The coefficients are slightly shrunk, making the model more robust. Cross-validation confirmed that performance is stable across different data subsets.

3.3 Lasso Regression Implementation with Cross-Validation

Lasso regression uses L1 regularization, which can shrink coefficients all the way to zero, effectively performing automatic feature selection. This creates sparser models that are easier to interpret. Cross-validation is used for optimal alpha selection.

3.3.1 Define Alpha Values

Why: Similar to Ridge, we need to test a range of alpha values to find the optimal regularization strength. Lasso typically requires smaller alpha values than Ridge.
What was done: A logarithmically spaced array of 100 alpha values from 0.0001 to 0.1 was created, focused on the smaller range where Lasso typically performs best.

3.3.2 Cross-Validated Alpha Selection with LassoCV

Why: Cross-validation provides robust alpha selection and helps identify which features to eliminate for different regularization strengths.
What was done: LassoCV was used to perform 5-fold cross-validation across all alpha values. The process automatically selected the optimal alpha. Visualization showed how both MAE and the number of selected features change with alpha.
Finding: The best alpha was identified (typically around 0.0001-0.0003), indicating that very light regularization was optimal. The feature selection plot showed that as alpha increases, more features are eliminated.

3.3.3 Final Lasso Model Results

Results:

- RMSE: ~€2,880 (very similar to baseline €2,879)
- MAE: ~€2,002 (identical to baseline)
- R^2 Score: ~0.8276 (marginally lower than baseline 0.8282)

Feature elimination: Lasso set several feature coefficients to exactly zero, identifying features with negligible predictive value and creating a simpler, more interpretable model.
Insight: The very small optimal alpha value (~ 0.0002) indicates minimal regularization was needed, confirming the baseline model wasn't overfitted. Cross-validation demonstrated stable feature selection and consistent performance across folds.

3.4 Regularization Comparison & Analysis

3.4.1 Metrics Comparison

Why: Comparing all three models side-by-side reveals whether regularization provided meaningful improvements and helps select the best model for deployment.

What was done: A comparison table was created showing RMSE, MAE, and R^2 for all three models (Baseline Linear Regression, Ridge, and Lasso).

Model	RMSE (€)	MAE (€)	R^2 Score
Baseline Linear Regression	2,879	2,002	0.8282
Ridge Regression	2,876	2,001	0.8279
Lasso Regression	2,880	2,002	0.8276

Analysis: The performance differences are minimal (within 0.1%). Ridge shows slight edge with lowest RMSE, but all three models are essentially equivalent in predictive accuracy.

3.4.2 Coefficient Comparison

Why: Examining how regularization affects individual feature coefficients provides insights into model stability and feature importance.

What was done: Coefficients from all three models Are merged and compared, particularly focusing on the largest coefficients and features eliminated by Lasso.

Key observations:

- Top positive coefficients (increase price): hp_kW remained strongest across all models, followed by Gears and weight_kg
- Top negative coefficients (decrease price): age and km showed consistent strong negative effects
- Regularization effect: Ridge and Lasso coefficients are slightly smaller in magnitude compared to baseline (e.g., hp_kW: 0.179 baseline vs 0.176 Ridge vs 0.178 Lasso)
- Feature elimination: Only Lasso zeroed out 3 features, creating a more parsimonious model

4. Conclusion & Key Takeaways

4.1 Overall Findings

All three models (Linear Regression, Ridge, and Lasso) achieved remarkably similar performance with R^2 scores around 0.82-0.83 and RMSE of approximately €2,870-2,880. The minimal performance difference indicates the baseline model was not suffering from severe overfitting, which is why regularization provided only marginal improvements.

4.2 Impact of Regularization

Ridge Regression (alpha = 87.3):

- Shrunk coefficients uniformly while retaining all 81 features
- Reduced sensitivity to multicollinearity among engine-related features
- Provided marginal improvement over baseline (RMSE: 2,876 vs 2,879)

Lasso Regression (alpha = 0.0002):

- Performed automatic feature selection by eliminating 3 features with negligible predictive value
- Created a sparser, more interpretable model with 78 features instead of 81
- Achieved similar performance to Ridge but with improved model simplicity

4.3 Overfitting Assessment

No significant overfitting detected. Training and test metrics are very close across all models (training RMSE: 2,866 vs test RMSE: 2,879), indicating excellent generalization. The large dataset (15,915 samples) and effective feature engineering prevented overfitting despite having 81 features.

The very small optimal alpha values for both Ridge (87.3) and Lasso (0.0002) confirm that heavy regularization was unnecessary, as the baseline model was already Ill-balanced.

4.4 Data Sufficiency

The dataset is sufficient and Ill-structured:

- Adequate sample size (15,915 records) relative to number of features (81)
- Strong predictive features identified: horsepower, age, mileage, fuel type, body type
- Excellent data quality with zero missing values and minimal outliers

Potential improvements: While the current features explain 82-83% of price variance, additional features like brand reputation scores, market trends, or seasonal effects could potentially capture the remaining 17-18% of unexplained variance.

4.5 Linear Model Adequacy

Linear regression is sufficient for this task. Achieving 82-83% R^2 demonstrates that the relationship between features and log-transformed price is predominantly linear. While non-linear models (Random Forest, XGBoost, Neural Networks) might achieve 2-5% better accuracy, the interpretability and simplicity of linear models make them ideal for business applications where stakeholders need to understand pricing factors.

The log transformation of the target variable was crucial in linearizing the relationship and meeting regression assumptions, as confirmed by the residual analysis.

4.6 Key Business Insights

Top Price Drivers (Positive Impact):

- Horsepower (hp_kW): Strongest predictor - each additional kW significantly increases value
- Number of Gears: More gears indicate newer, more advanced transmissions
- Diesel Fuel: Diesel vehicles command premium pricing in this market
- weight: Heavier vehicles typically more expensive (SUVs, luxury cars)
- Premium Features: Leather upholstery, automatic transmission, 4WD/rear-wheel drive all add value

Price Reducers (Negative Impact):

- Age: Each additional year significantly decreases value
- Mileage (km): Higher usage reduces price substantially
- Manual Transmission: Manual gearboxes less desirable than automatic
- Displacement: Larger engines correlate with older, less efficient vehicles

Lasso-Eliminated Features: Parking assist camera, MP3 systems, and station wagon body type showed negligible predictive value, likely because they're either too common (MP3) or too rare (parking camera) to differentiate prices.

4.7 Business Recommendations

- **Model Deployment:** Deploy the Ridge or Lasso model (both perform equally ll) for automated price estimation in your online platform
- **Inventory Strategy:** Prioritize acquiring diesel vehicles with automatic transmission, higher horsepower, and premium features as they command better margins
- **Pricing Guidance:** Use the model coefficients to justify pricing decisions to customers - quantify how age, mileage, and features affect value

- **Feature Focus:** When photographing and describing vehicles, emphasize horsepower, low mileage, recent year, and premium features
- **Market Positioning:** The model explains 82-83% of price variance - the remaining variation likely comes from brand perception, condition details not captured, and negotiation factors
- **Model Monitoring:** Retrain the model quarterly with new sales data to capture evolving market trends and seasonal effects

4.8 Technical Conclusion

The analysis successfully developed a robust car price prediction model with 82-83% accuracy. The minimal benefit from regularization indicates excellent baseline model quality achieved through:

- Proper feature engineering and encoding strategies
- Appropriate handling of categorical variables with imbalanced classes
- Effective log transformation of the target variable
- Adequate sample size relative to feature count
- Feature scaling and standardization

While the three models show nearly identical performance, Lasso regression is recommended for deployment due to its automatic feature selection creating a more parsimonious and interpretable model without sacrificing accuracy.

--- End of Report ---